

EYERISS-v2 CNN ACCELERATOR

RTL Architecture Schematic — Professional Reference

Team: SMC26-30 | Target: LEGNet on FPGA | Rev B2 | 2025-01

1. SYSTEM OVERVIEW

Architecture	Eyeriss-v2 hierarchical mesh NoC CNN accelerator
Dataflow	Row-stationary with weight reuse across PE columns
PE Array	9 rows × 4 columns = 36 Processing Elements
Precision	8-bit INT activations × 8-bit weights → 20-bit psum
Compression	CSC (Compressed Sparse Column) for zero-skipping
Interfaces	DDR4/HBM external memory via mem_data[255:0] bus
Clocking	Single clock domain (clk), async active-low rst_n
Completion	IRQ pulse on done; FSM: IDLE→CONFIG→RUN→DRAIN→DONE

2. SIGNAL BUS LEGEND

Bus	Width / Color	Description
iact_bus[7:0]	8-bit / Blue (4A90D9)	Input activation data from GLB iact SRAM → PE iSPad
wt_bus[7:0]	8-bit / Green (2E8B57)	Weight data from GLB weight SRAM → PE wSPad
psum_bus[19:0]	20-bit / Rose (C0392B)	Partial sums PE↔PE and PE→GLB psum SRAM (signed int20)
clk / rst_n	1-bit / Amber	Global clock (posedge) and async active-low reset

3. TOP-LEVEL PORT MAP

Port Name	Direction / Width / Description
clk	input 1b – Global clock, posedge triggered
rst_n	input 1b – Async active-low reset
start	input 1b – Begin inference (1-cycle pulse)
done	output 1b – Inference complete (1-cycle pulse)
layer_cfg[31:0]	input 32b – Layer configuration register
mem_data[255:0]	bidir 256b – DDR4/HBM data bus
mem_addr[31:0]	output 32b – Memory address output
mem_req	output 1b – Memory request strobe
mem_ack	input 1b – Memory acknowledge
mem_we	output 1b – Memory write enable
irq	output 1b – Interrupt: inference complete

4. MODULE DESCRIPTIONS

OFF-CHIP MEMORY (DDR4 / HBM)

Interface: mem_data[255:0] | mem_addr[31:0] | mem_req | mem_ack | mem_we
Bandwidth: 256-bit wide burst transfers

Protocol: Request/Acknowledge handshake with CGC DMA engine

CLUSTER GROUP CONTROLLER (CGC)

Sub-modules:

addr_gen: Generates DMA addresses for each layer's iact/weight tiles
dataflow_sched: Schedules PE rows/cols for row-stationary mapping
sync_ctrl: Synchronises GLB ↔ Router ↔ PE Array pipeline
layer_cfg: Holds 32-bit per-layer config: kernel size, strides, padding

Control signals:

Inputs: clk rst_n start layer_cfg[31:0] instr[63:0] mem_ack
Outputs: done irq mem_req mem_we mem_addr[31:0] cluster_id[3:0]
FSM: IDLE → CONFIG → RUN → DRAIN → DONE → IDLE
Handshake: valid/ready on all data interfaces; IRQ on completion

GLOBAL BUFFER (GLB) — On-chip SRAM Banks

iact SRAM

Depth: 256 entries
Width: 8-bit
Read ports: 3 simultaneous
Write port: 1 | Latency: 3 cy
Signals: read_addr[7:0] read_en
: read_done wr_data[7:0]
CSC: Run-length encoded input

weight SRAM

Depth: 256 entries
Width: 8-bit
Read ports: 3 | Write: 1 (lat 3cy)
Output: Circuit-switch MUX → PE cols
CSC: run_len[3:0] | value[7:0]

psum SRAM

Depth: 32 entries
Width: 20-bit signed int
Ports: 1 read | 1 write
Mode: Accumulate: psum += MAC_out
IRQ: irq_psum_full on overflow

Read timing:

1. Set read_addr[4:0]
2. Assert read_en (1 cycle)
3. Data valid after 2-cy lat
4. read_done pulses on end

CSC Encoding Detail:

Format: [run_len[3:0]|value[7:0]]
Example: [0,0,0,3,0,7]
→ [3:03][1:07] (2 vs 6 entries)
Ratio: up to 4× compression

ROUTER CLUSTER — iact / weight / psum

iact Router

Mode 0: UNICAST → single PE
Mode 1: MULTICAST → up to 6 PEs
Mode 2: HOR_CAST → full row
Mode 3: VER_CAST → full column
Mode 4: BROADCAST → all PEs
Arb: Round-robin priority
Buffer: 2-deep FIFO per output
HS: valid_in/ready_in

weight Router

Type: Circuit-switch crossbar MUX
Enable: col_en[3:0] from CGC
Modes: Broadcast / per-column
Config: mux_sel[1:0] from CGC

psum Router

Type: Reduction tree accumulator
Links: North ↔ South inter-PE
Width: Signed 20-bit adder trees
Output: psum → GLB psum SRAM
Flag: overflow_flag output
HS: valid/ready on all ports

NoC Topology

Mesh: Hierarchical 2D mesh
VC: Virtual channels per type
Credit: Credit-based flow control
Route: XY deterministic routing

5. PE ARRAY — $9 \times 4 = 36$ PEs

Processing Element (PE) Internal Structure

Per-PE Scratchpads:

iSPad: $16 \times 8b$ — input activations
wSPad: $9 \times 8b$ — weights (filter)

iact feed: Tap from horizontal row bus
wt feed: Vertical column bus (top)

pSPad: 16 × 20b – partial sums

psum: Downward chain to collect bus

MAC Unit

Operation: $iact[7:0] \times wt[7:0] + psum[19:0] \rightarrow psum[19:0]$

Arithmetic: $8b \times 8b + 20b = 20b$ signed integer

Latency: 1 cycle, fully pipelined

Overflow: Saturation clamping to $\pm(2^{19}-1)$

Accumulate: N MAC cycles per output activation

Status pins: busy | stall | mac_valid

PE FSM States:

IDLE	LOAD_WT	LOAD_IACT	MAC_COMPUTE	PSUM_DRAIN	FLUSH
Wait start	Load wSPad	Load iSPad	Accumulate	Send psum	Flush regs

FSM signals: state[2:0] done stall mac_valid valid/ready on all SPad ports

Loop back: FLUSH → IDLE on done assertion or stall timeout

6. DATAFLOW PIPELINE

#	Source	Route	Destination	Dtype	Notes
1	GLB weight SRAM	weight Router	wSPad (all PEs)	8-bit INT	Stationary; broadcast to all cols before iact stream
2	GLB iact SRAM	iact Router	iSPad (per row)	8-bit INT	Multicast per row; 1 row per cycle; pipelined
3	iSPad + wSPad	MAC Unit	pSPad (20b)	20b signed	1-cycle latency; accumulate N cycles per OFM element
4	pSPad (each PE)	psum Router	GLB psum SRAM	20b signed	Column reduction tree; drain after MAC completes

7. PE ARRAY CONNECTIVITY GRID

Each cell = PE[row,col]. Colors: iact bus (row) ← left | wt bus (col) ↑ top | psum ↓ bottom.

Row \ Col	Col 0 wt_col0[7:0]↓	Col 1 wt_col1[7:0]↓	Col 2 wt_col2[7:0]↓	Col 3 wt_col3[7:0]↓
Row 0 iact_r0→	PE[0,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[0,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[0,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[0,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 1 iact_r1→	PE[1,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[1,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[1,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[1,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 2 iact_r2→	PE[2,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[2,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[2,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[2,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 3 iact_r3→	PE[3,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[3,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[3,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[3,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 4 iact_r4→	PE[4,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[4,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[4,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[4,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 5 iact_r5→	PE[5,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[5,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[5,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[5,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 6 iact_r6→	PE[6,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[6,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[6,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[6,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 7 iact_r7→	PE[7,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[7,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[7,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[7,3] iSPad·wSPad·pSPad MAC 8b×8b+20b
Row 8 iact_r8→	PE[8,0] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[8,1] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[8,2] iSPad·wSPad·pSPad MAC 8b×8b+20b	PE[8,3] iSPad·wSPad·pSPad MAC 8b×8b+20b

All interfaces use valid/ready handshake. Signed arithmetic throughout psum path. CSC encoding reduces off-chip bandwidth up to 4x. Row-stationary dataflow maximises iact reuse per PE row.